Milestone 3: A Fast, Verified 7-Card Hand Evaluator

holdem-cfr walkthrough

Why the evaluator matters, and where it does not

Every hold'em showdown asks the same question: given seven cards, how strong is the best five-card hand? The abstraction work in milestone 4 asks it billions of times. Computing expected hand strength for every flop hand means rolling out turns and rivers and comparing each hand against every possible opponent hand. Monte Carlo CFR rollouts in milestone 5 ask it again at every sampled showdown.

The evaluator matters less than it first appears inside the solver itself, though. Johanson et al. (2011) observe that on a fixed river board you can rank all 1,326 hole-card pairs once, sort them, and then settle every showdown in the betting tree with running sums. Raw evaluation speed is therefore mainly a cost of building the abstraction, and correctness is what matters everywhere. That is why this milestone spends more lines on tests than on the evaluator.

Equivalence classes: the number 7,462

There are 2,598,960 five-card hands, but many tie. A-K-Q-J-9 of hearts and the same ranks in spades are equal; so are any two "two pair, kings and fives, queen kicker" hands. Poker strength only cares about the multiset of ranks plus one bit: whether all five cards share a suit. Grouping hands that always tie leaves exactly **7,462 equivalence classes**, which split by category as 1,277 high-card, 2,860 one-pair, 858 two-pair, 858 trips, 10 straights, 1,277 flushes, 156 full houses, 156 quads and 10 straight flushes.

The evaluator returns a `HandRank(u16)` from 1 (seven-high: 7-5-4-3-2 offsuit) to 7,462 (the royal flush). Higher wins and equal ranks split. Using a dense rank rather than an ad hoc score has two benefits. It fits in 16 bits, and the category can be read off from fixed boundaries, since every pair ranks above every high card and so on.

The reference evaluator: slow on purpose

Before writing anything fast we need ground truth, and ground truth should be too simple to be wrong. `reference::eval5_ranks` counts how many cards there are of each rank. It sorts the (count, rank) groups by count then rank, checks for a straight (including the wheel A-2-3-4-5, where the ace plays low and the hand is five-high), and classifies the hand by its shape. Shape `[4, 1]` is quads, `[3, 2]` is a full house, `[2, 2, 1]` is two pair, and so on. The result packs the category into the top bits and the

ordered kickers below, so comparing two `u32` values compares the hands. For six or seven cards, `eval_best` tries all 21 five-card subsets and keeps the best. Nothing about it is clever, which is the point.

How do we know the reference itself is right? Poker hand frequencies are textbook combinatorics. The first test runs the reference over all 2,598,960 five-card hands and requires the category counts to be exactly 1,302,540 high card, 1,098,240 one pair, 123,552 two pair, 54,912 trips, 10,200 straights, 5,108 flushes, 3,744 full houses, 624 quads and 40 straight flushes. It also requires the number of distinct values per category to match the class counts above. An off-by-one in straight detection or kicker ordering would break at least one of these eighteen numbers.

The fast evaluator: two kinds of hand, two tables

The design follows Henry Lee's perfect-hash evaluator, which is what the paper search recommended for Rust. It starts from one observation about seven cards. If five or more share a suit, the best hand is a flush or straight flush, and it depends only on *which ranks* that suit holds. Quads and full houses are impossible alongside a flush in seven cards. A full house needs three suits for the trips and two for the pair, so at least three of the seven cards would have to sit outside the flush suit, but only two do. If no suit has five cards, suits are irrelevant and the answer depends only on how many cards of each rank there are.

So the hand is stored two ways at once, and both are maintained incrementally:

```
pub struct Hand {
    suits: u64,   // bit 16*suit + rank: four 13-bit masks, one per suit
    counts: u64,  // 4 bits per rank: how many of that rank we hold
    suit_counts: u32, // 1 byte per suit: 3 + cards of that suit (see "Speed")
    len: u32,
}

pub fn plus(self, card: Card) -> Hand {
    let bit = 1u64 << (16 * card.suit() as u32 + card.rank() as u32);
    Hand {
        suits: self.suits | bit,
        counts: self.counts + (1u64 << (4 * card.rank() as u32)),
        suit_counts: self.suit_counts + (1u32 << (8 * card.suit() as u32)),
        len: self.len + 1,
    }
}
```

Adding a card is an OR and three adds. `Hand` is `Copy` and 24 bytes, so a board is folded in once and each player's hole cards are added on top of a copy. That pattern is exactly what equity enumeration needs.

Flushes. If any suit holds five or more cards (detected with one AND, explained under Speed below), the answer is `flush[mask]`, a direct table of 8,192 entries (2^{13})

indexed by the mask itself.

Everything else. The packed counts form a vector (c_0, ..., c_{12}): one entry per rank, each between 0 and 4, summing to n. There are only 49,205 such vectors for n = 7, but the packed integer can be as large as 2^{52}, far too big to index directly. We need a **perfect hash**: a function that maps every valid vector to a distinct small integer with no gaps. The combinatorial number system gives one. Order the vectors lexicographically. The index of a vector is then the number of vectors that come before it. Walking ranks from 0 to 12 with k cards still to place, each rank contributes the number of vectors that agree on the earlier ranks but put *fewer* cards on this one. Let N(m, j) be the number of length-m vectors with entries 0 to 4 that sum to j. Putting count c at rank r then contributes

$$\text{STEP}[r][k][c] = N(12 - r, k) + N(12 - r, k - 1) + \ldots + N(12 - r, k - c + 1)$$

Both N and STEP are computed at *compile time* with `const fn`. That is Rust's way of running ordinary code inside the compiler, so the tables are baked into the binary:

```
pub(crate) fn nonflush_index(counts: u64, n: u32) -> usize {
    let mut k = n as usize;          // cards still to place
    let mut idx = BASE[k];
    for (r, step) in STEP.iter().enumerate() {
        let c = ((counts >> (4 * r)) & 0xF) as usize;
        idx += step[k][c];
        k -= c;
    }
    idx as usize
}
```

The loop always runs all 13 ranks. Once every card is placed, k is 0 and the remaining entries add nothing; the Speed section explains why the loop has no early exit. `BASE` puts the 5-, 6- and 7-card sections (1,287 + 6,175 + 49,205 entries) side by side in one table, so the same evaluator handles 5, 6 and 7 cards. All the tables together come to about 125 KB, small enough to stay in the CPU cache. The two-plus-two evaluator instead uses about 130 MB, which is faster for perfectly sequential enumeration but cache-hostile for the random access that sampling produces.

Building the tables from the reference

The fast tables are not typed in by hand. They are *generated from the reference* the first time they are used, which takes a few milliseconds. The builder first collects every 5-card class value and sorts them, giving the dense 1..7,462 mapping. It then enumerates every count vector for n = 5, 6 and 7 and stores the dense rank of its best non-flush five at `nonflush_index(vector)`. Last, it enumerates every flush mask and does the same with the flush flag set.

The tables live in a `static` behind `std::sync::OnceLock`. That is a thread-safe "initialize exactly once" cell: the first caller builds the tables, any concurrent caller waits, and every later call is a single atomic load. This matters for milestone 5, when many threads will evaluate hands at once.

The test pyramid

Generating the fast tables from the reference means the reference is the single source of truth, so the tests attack from several directions:

Test	Scope	What it pins down
reference counts	all 2,598,960 five-card hands	reference matches textbook category and class counts
fast vs reference	all 2,598,960 five-card hands	fast equals reference everywhere; exactly 7,462 distinct ranks
random 6 and 7 cards	600,000 hands	fast equals brute-force best-of-21
7-card histogram	all 133,784,560 hands (CI)	category counts match the known 7-card frequencies; exactly 4,824 distinct ranks
full brute force	all 133,784,560 hands (`--ignored`)	fast equals brute force on every 7-card hand
edge cases	handpicked	wheel, steel wheel, kickers, split pots, three pairs, two sets, quads plus trips

The 7-card histogram test is the one that ties everything to the outside world. Its numbers are standard combinatorics: 23,294,460 high card, 58,627,800 one pair, 31,433,400 two pair, 6,461,620 trips, 6,180,020 straights, 4,047,644 flushes, 3,473,184 full houses, 224,848 quads and 41,584 straight flushes. None of them were produced by our code. The test also checks a less well known fact: only **4,824** of the 7,462 classes can be the best five of seven cards. With seven cards you always have two spare, so a hand like 7-5-4-3-2 high can never be your best. The histogram test enumerates all 133 million hands across threads in about two seconds, so it runs in CI on every push.

The full brute-force comparison makes about 2.8 billion reference evaluations, so it is marked `#[ignore]` and runs on demand:

```
cargo test --release -p eval -- --ignored
```

On the development machine, a 2-core cloud VM, it finished in 110 seconds with **zero mismatches across all 133,784,560 hands**. That is the strongest statement

this milestone can make: the fast evaluator agrees with the obviously correct one on every seven-card hand that exists.

A note on Rust: scoped threads

The exhaustive tests parallelize over the lowest card. A shared atomic counter hands out "first card" values 0 to 45 to however many threads the machine has:

```
let next = AtomicUsize::new(0);
std::thread::scope(|s| {
    for _ in 0..threads {
        s.spawn(|| loop {
            let first = next.fetch_add(1, Ordering::Relaxed);
            if first > 45 { break; }
            // enumerate every hand whose lowest card is `first`
        });
    }
});
```

`std::thread::scope` guarantees that every spawned thread finishes before the scope returns. That lets the threads *borrow* local variables such as `next` and the closure, with no `Arc` and no `'static` requirement. The borrow checker proves this is safe, and that is why the code reads like single-threaded code. `Ordering::Relaxed` is enough here because the counter only has to hand out each value once; it does not publish any other data. In milestone 5 we will graduate to `rayon` for parallel CFR traversals, but the idea of splitting the work and borrowing shared read-only data is the same.

Speed

Measured single-threaded on a shared 2-core cloud VM. Expect noise of about 10%.

Benchmark	Throughput
rank a prebuilt random 7-card `Hand`	~62M hands/s
build from 7 random cards, then rank	~35M hands/s
build from 5 random cards, then rank	~40M hands/s
enumerate every 7-card hand containing the 2c (criterion)	~54M hands/s
enumerate all 133,784,560 hands, plus category (`bench-eval`)	~45–53M hands/s

Two optimizations got us here, and one idea did not.

The first version detected flushes with four popcounts in a loop and walked the hash with an early exit, which ran at about 40M/s on random hands. **Win one:** keep a fourth counter with one byte per suit, each starting at 3. Five cards of a suit make a

byte reach 8, which sets bit 3, so a single `suit_counts & 0x08080808` detects any flush with no loop at all. **Win two:** make the hash walk always run all 13 steps. Once every card is placed, the remaining entries add 0. That removes a branch the CPU could not predict on random hands and lets the compiler unroll the loop completely. Together these gave a 54% speedup on random hands.

The idea that failed is worth recording. The walk has a dependency chain: the number of cards still to place, k, at rank r depends on every earlier rank. A SWAR trick removes the chain. Multiplying the packed counts by 0x1111111111111 puts, in each nibble, the total count of all ranks up to that one. That makes all 13 table loads independent. In theory that is better for an out-of-order CPU. In practice it measured about 20% *slower*, presumably because the extra shifts and masks cost more than the short, cache-resident chain they replaced. We reverted it. Measure, don't guess.

As a rough guide: random access runs at tens of millions of hands per second, and incremental enumeration, where the six-card prefix is shared, runs faster still. Your M-series MacBook Air should beat these cloud numbers comfortably. Run `cargo bench -p eval` for the criterion report, or `cargo run --release -p cli -- bench-eval` for the end-to-end single-thread number.

Is that fast enough? Computing exact E[HS] for one flop hand means, for each of 1,081 turn-and-river runouts, comparing against up to 990 opponent hands. That is about a million evaluations per flop hand, or a few hundredths of a second. With 1.3 million canonical flop hands after Waugh's isomorphism (milestone 4), the full flop table is minutes of work across both cores, not hours. Sorting hand ranks on each river board removes most of the repeated work anyway.

Try it

```
cargo run --release -p cli -- eval "As Ks Qs Js Ts"      # straight flush (7462)
cargo run --release -p cli -- eval "Ah 2h 3h 4h 5h 9c 9d" # straight flush (7453), the steel wheel
cargo run --release -p cli -- bench-eval
```

Reading

Henry Lee's algorithm write-up for the perfect hash (https://github.com/HenryRLee/PokerHandEvaluator, Documentation/Algorithm.md). Cactus Kev's page on the 7,462 classes (http://suffe.cool/poker/7462.html). Johanson et al. (2011), section 4, for why sorted ranks make showdowns linear-time. That is the next performance lever after this one.